

内网加密文件传输系统

软件开发文档 (SDD) · v0.2.0

内网加密文件传输系统 —— 软件开发文档 (SDD)

版本：v0.2.0

文档日期：2026-09-20

项目路径：`secure-transfer/`

1. 项目概述

1.1 背景与目标

在内网环境中，团队需要在成员之间安全地传输文件。传统方案（共享盘、FTP、即时通讯）存在明文落盘、无权限控制、无审计等问题。本系统目标是提供一个**端到端加密（E2EE）**的文件传输 Web 应用，核心原则：

****服务端永不接触文件明文，也永不接触明文 fileKey。****

1.2 目标用户与规模

- 团队规模：10-50 人
- 使用方式：浏览器访问内网 Web 页面（无需安装客户端）
- 部署形态：单机部署（一台内网服务器）

1.3 核心能力

能力	说明
-----	-----
端到端加密	文件在浏览器端用 AES-256-GCM 加密，服务端只存密文
多接收方共享	同一文件可安全共享给多个用户，各自独立解封
身份认证	Argon2id 口令哈希 + JWT
操作审计	全量操作日志，仅管理员可见，支持搜索/分页
生命周期管理	文件有效期（到期自动物理删除）、手动删除、撤销访问
防投毒	文件类型白名单 + 真实类型（魔数）检测 + 压缩包递归扫描
管理员免审	管理员可上传任意类型文件（背书制）

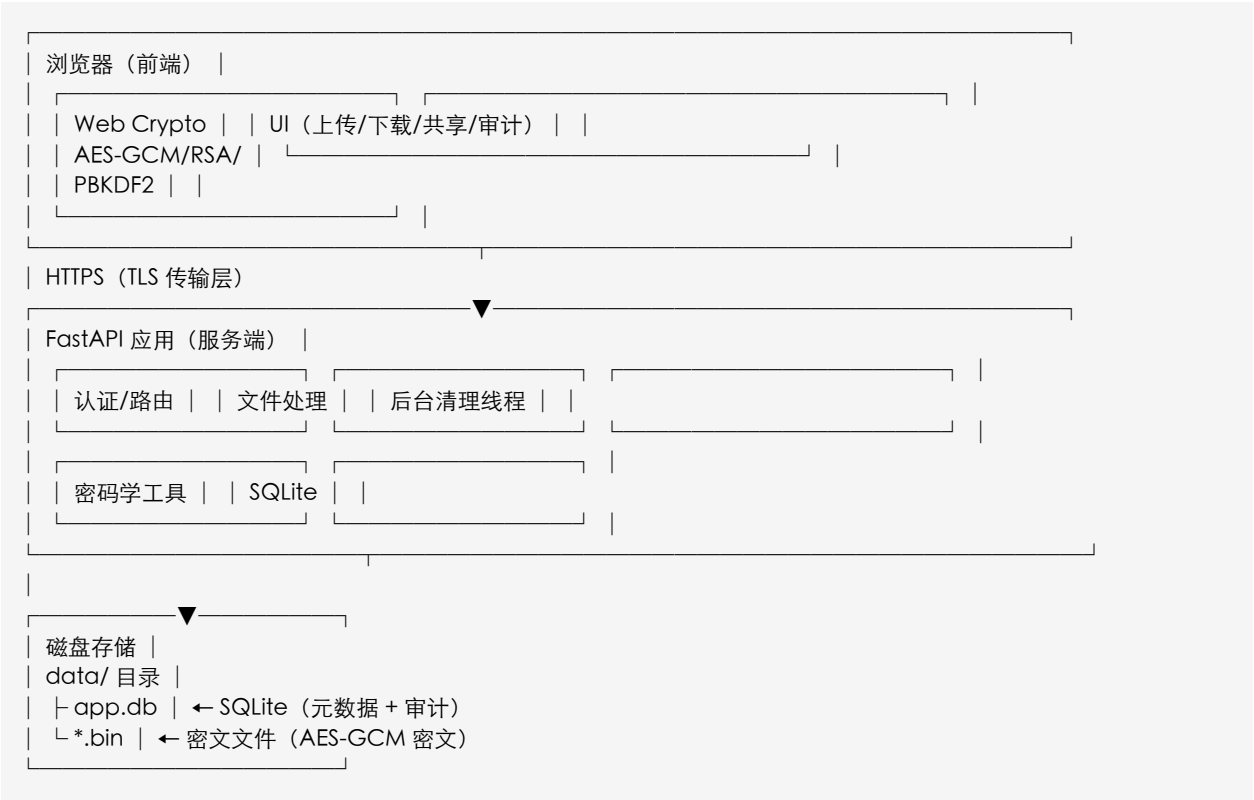
2. 技术选型

层	技术	版本	理由
----	-----	-----	-----
后端框架	FastAPI	0.115.6	异步、类型安全、自动文档
ASGI 服务器	Uvicorn	0.34.0	轻量、支持 TLS
数据库	SQLite (WAL)	内置	单机内网场景足够，零运维

层	技术	版本	理由
口令哈希	Argon2id	argon2-cffi 23.1.0	抗 GPU 暴力破解
令牌	PyJWT (HS256)	2.10.1	无状态认证
后端加密	cryptography	44.0.0	RSA 封装/解封
前端加密	Web Crypto API	浏览器原生	无需引入加密库
前端压缩解包	fflate	本地静态文件	zip 递归扫描
配置	pydantic-settings	2.7.0	.env 环境变量管理

3. 系统架构

3.1 逻辑架构



3.2 目录结构

```
secure-transfer/
├── config.py # 配置 (pydantic-settings, 读 .env)
├── run.py # 启动入口 (HTTPS + HTTP→HTTPS 跳转)
├── test_e2e.py # 端到端集成测试
├── requirements.txt # 依赖
├── .env # 环境变量 (JWT_SECRET、TLS、管理员等)
├── server.crt/key # TLS 自签证书
├── app/
│   ├── __init__.py
│   ├── main.py # 路由 + 业务逻辑 + 后台清理线程
│   ├── db.py # SQLite 持久化层
│   └── security.py # Argon2id + JWT + RSA 封装
├── static/
│   ├── index.html # 前端 (Web Crypto E2EE + UI)
│   └── fflate.min.js # zip 递归扫描库 (本地静态文件)
├── data/ # 运行时数据 (密文 + app.db)
├── app.db
└── <uuid>.bin
```

3.3 运行模式

- 主服务：HTTPS 监听 `settings.port`（默认 443）
- 跳转服务：HTTP 监听 `settings.http_port`（默认 80），任意请求 301 → HTTPS
- 后台线程：`purge-expired`，每 `purge_interval_seconds`（默认 60 秒）扫描并物理删除过期文件

4. 安全模型与密码学设计

4.1 密码学参数汇总

用途	算法	参数
-----	-----	-----
文件内容加密	AES-256-GCM	256 位随机 fileKey，随机 96 位 IV
口令 → KEK 派生	PBKDF2	SHA-256，310,000 次迭代，随机盐
KEK → 私钥封装	AES-256-GCM	KEK 加密 RSA 私钥
用户密钥对	RSA-2048	OAEP 填充，SHA-256
fileKey 分发封装	RSA-OAEP(SHA-256)	每个有权用户独立封装一份
口令哈希	Argon2id	t=3, m=65536KB(64MB), p=4（见 8.1 待加固项）
认证令牌	JWT	HS256，默认 12 小时过期
传输层	TLS	自签证书（内网）

4.2 端到端加密核心流程

4.2.1 用户注册

1. 浏览器本地生成 RSA-2048 密钥对
2. 用登录口令经 PBKDF2(310000, SHA-256) 派生 KEK
3. 用 KEK (AES-GCM) 加密 RSA 私钥 → private_key_wrapped
4. 上传到服务端：public_key(明文) + private_key_wrapped + kek_salt + kek_iv
5. 服务端用 Argon2id 哈希口令，存 password_hash

服务端只存「加密后的私钥」和「Argon2 哈希后的口令」，永不接触明文私钥和明文口令。

4.2.2 文件上传（E2EE）

1. 浏览器生成随机 fileKey (AES-256) 和随机 IV
2. 用 fileKey + IV 加密文件 → 密文 (AES-256-GCM)
3. 用自己的 RSA 公钥 OAEP 封装 fileKey → wrapped_key
4. 上传：密文文件 + wrapped_key + iv + 元数据
5. 服务端落盘密文，存元数据到 files 表，wrapped_key 到 file_keys 表

4.2.3 文件下载（E2EE）

1. 请求下载 → 服务端校验权限，流式返回密文 + 元数据（含 wrapped_key）
2. 浏览器用登录口令派生 KEK → 解密私钥（进内存）
3. 用私钥 RSA-OAEP 解封 fileKey
4. 用 fileKey + iv 解密密文 → 明文（浏览器端完成）

4.2.4 多接收方共享

1. Owner 从自己元数据解封 fileKey（需自己的私钥）
2. 用接收方公钥 RSA-OAEP 重新封装 fileKey → 新的 wrapped_key

3. 调用 share 接口，把新的 wrapped_key 存入 file_keys 表（接收方条目）
4. 接收方登录后用自己私钥解封自己的 wrapped_key → 解密文件

关键设计：******每个有权用户有自己独立封装的 fileKey******，即使某个用户口令泄露，只影响该用户名下文件，不波及其他用户。

4.3 安全边界（诚实声明）

威胁	防护状态
-----	-----
传输层窃听	TLS 加密
服务端窥探文件明文	E2EE，服务端只有密文
服务端窥探 fileKey	服务端只有 RSA-OAEP 封装后的 wrapped_key
数据库泄露解密文件	只能靠暴力破解口令（安全锚点=口令强度）
口令暴力破解	Argon2id t=3 偏低，建议 t≥10
忘记口令	私钥永久无法恢复（需 KMS 托管方案）
恶意文件（投毒）	白名单 + 魔数检测 + zip 递归扫描
绕过前端直调 API	服务端只能看密文，无法解密，但可上传垃圾数据

5. 数据库设计（SQLite）

5.1 表结构

users（用户表）

字段	类型	说明
-----	-----	-----
id	INTEGER PK	自增
username	TEXT UNIQUE	用户名
password_hash	TEXT	Argon2id 哈希
public_key	TEXT	base64(SPKI) RSA 公钥
private_key_wrapped	TEXT	base64(AES-GCM 加密的 PKCS8 私钥)
kek_salt	TEXT	base64(PBKDF2 盐)
kek_iv	TEXT	base64(私钥封装 AES-GCM IV)
created_at	TEXT	创建时间

files（文件表）

字段	类型	说明
-----	-----	-----
id	TEXT PK	uuid4 hex
owner_id	INTEGER FK	属主用户
filename	TEXT	原始文件名
stored_name	TEXT	磁盘密文文件名（<id>.bin）
size	INTEGER	密文字节数

字段	类型	说明
original_size	INTEGER	明文字节数
iv	TEXT	文件 AES-GCM IV (base64)
created_at	TEXT	创建时间
expires_at	TEXT	过期时间（可空）

file_keys（访问密钥表，多接收方核心）

字段	类型	说明
-----	-----	-----
file_id	TEXT FK	文件
user_id	INTEGER FK	有权用户
wrapped_key	TEXT	RSA-OAEP 加密的 fileKey (base64)
created_at	TEXT	创建时间

主键 `(file\_id, user\_id)`，即「每个文件 × 每个有权用户」一条记录，存储该用户专属的 wrapped_key。

audit_log（审计日志表）

字段	类型	说明
-----	-----	-----
id	INTEGER PK	自增
user_id	INTEGER	操作人 id
username	TEXT	操作人用户名
action	TEXT	register/login/upload/download/share/revoke/delete/remove_access
file_id	TEXT	涉及文件（可空）
filename	TEXT	文件名（可空）
target	TEXT	目标（共享/撤销对象，可空）
ip	TEXT	来源 IP
created_at	TEXT	时间

5.2 索引

- `idx_file_keys_user` on `file_keys(user_id)`
- `idx_audit_created` on `audit_log(created_at)`

5.3 数据库模式

- `PRAGMA journal_mode=WAL`（写并发友好）
- `PRAGMA foreign_keys=ON`（外键约束）

6. API 接口文档

所有接口（除注册/登录外）需 `Authorization: Bearer <token>`。

6.1 认证

POST /api/auth/register

注册新用户（含 RSA 密钥对）。表单参数：

参数	说明
-----	-----
username	用户名
password	口令（≥8 位）
public_key	base64 SPKI 公钥
private_key_wrapped	base64 加密私钥
kek_salt	base64 PBKDF2 盐
kek_iv	base64 封装 IV

响应：{"ok": true, "id": <user_id>}

POST /api/auth/login

登录。表单：username, password。

响应：{"access_token": "...", "token_type": "bearer", "username": "..."}

GET /api/auth/me

返回当前用户信息（含私钥解密材料，不含私钥明文）。

响应字段：username, id, is_admin, public_key, private_key_wrapped, kek_salt, kek_iv

GET /api/users/search?q=

按用户名模糊搜索（共享选人用），返回 [{id, username, public_key}]。

6.2 文件

GET /api/files

列出当前用户可访问的文件（自有 + 被共享），已过期不展示。

返回数组，每项含 id, filename, size, original_size, created_at, expires_at, owner_id, owner_name, is_owner。

POST /api/files/upload

上传（multipart/form-data）：

参数	说明
-----	-----
file	密文文件
filename	原始文件名
iv	base64 文件 IV
wrapped_key	base64 上传者封装的 fileKey

参数	说明
original_size	明文大小
expires_at	可选 ISO 过期时间

响应：{"id": "<file_id>", "size": <密文大小>, "expires_at": ...}

GET /api/files/{file_id}/meta

文件元数据（含当前用户的 wrapped_key）。

GET /api/files/{file_id}/accesses

列出可访问该文件的用户（仅 owner）。

POST /api/files/share

共享。JSON body：{file_id, target_user_id, wrapped_key}。

POST /api/files/revoke

撤销访问。JSON body：{file_id, target_user_id}。

DELETE /api/files/{file_id}

删除：owner 物理删除（密文+元数据+所有密钥）；非 owner 移除自己访问权。

GET /api/files/{file_id}/download

下载密文（流式 FileResponse）。过期返回 410。

6.3 审计

GET /api/files/audit?q=&page=&page_size=

仅管理员。参数：

参数	说明
-----	-----
q	搜索关键字（匹配 username/ip/filename/action/target）
page	页码（默认 1）
page_size	每页条数（默认 50，上限 200）

响应：{"items": [...], "total": N, "page": 1, "page_size": 50}

6.4 健康检查

GET /healthz

响应：{"ok": true}

7. 前端设计

7.1 技术要点

- 单文件 static/index.html，原生 JS + Web Crypto API
- 拖拽上传 + 点击选择
- 所有加解密在浏览器端完成，服务端不可见明文

7.2 加密流程（前端）

- 1. `deriveKEK(passphrase, salt)` : PBKDF2(310000, SHA-256) → AES-GCM KEK
- 2. `generateKeypair()` : RSA-2048 OAEP 密钥对
- 3. `encryptFile(file)` : AES-256-GCM 加密 → `{cipher, fileKey, iv}`
- 4. `wrapFileKeyForRSA(fileKey, pubKey)` : RSA-OAEP 封装 fileKey
- 5. `decryptFile(cipher, fileKey, iv)` : 解密

7.3 文件类型防护（防投毒）

三层防护：

- 1. **后端扩展名白名单**（`config.py allowed\_extensions`，18 种）
- 2. **前端真实类型检测**：读文件头 64 字节魔数识别（防`.pdf`伪装`.exe`）
- 3. **zip 递归扫描**（flate 解包，5 层/1000 条目/512MB 上限防压缩炸弹）

管理员（<font face="Courier"

size="8.5">ADMIN_USERNAMES）上传时跳过扩展名校验，显示"管理员免审模式"提示。

7.4 UI 组件

- 登录/注册面板
- 文件列表（含过期状态、所有者、大小）
- 拖拽上传（含保存时间下拉：永久/1天/7天/30天/90天/自定义）
- 共享选择器（搜索用户）
- 审计模态框（搜索 + 分页 + 退出按钮，仅管理员）

8. 配置说明

8.1 环境变量（.env）

变量	默认值	说明
-----	-----	-----
HOST	0.0.0.0	监听地址
PORT	443	HTTPS 端口
JWT_SECRET	CHANGE_ME...	生产必须替换（openssl rand -hex 32）
JWT_EXPIRE_MINUTES	720	令牌有效期（分钟）
MASTER_KEY	空	服务端落盘主密钥（预留）
DATA_DIR	data/	密文存储目录
MAX_UPLOAD_MB	2048	单文件大小上限
ALLOWED_EXTENSIONS	见 config.py	扩展名白名单
ADMIN_USERNAMES	空	管理员用户名（逗号分隔）
PURGE_INTERVAL_SECONDS	60	过期清理扫描间隔
PURGE_ENABLED	true	是否启用过期自动清理
CERT_FILE	空	TLS 证书
KEY_FILE	空	TLS 私钥

变量	默认值	说明
HTTP_REDIRECT_ENABLED	true	HTTP→HTTPS 跳转
HTTP_PORT	80	跳转端口

8.2 已知待加固项

1. ****Argon2id t=3 偏低****：建议生产提升到 $t \geq 10$ （需同步更新已有用户哈希，或做登录时重哈希迁移）。
2. ****口令强度仅校验 ≥ 8 位****：建议强制复杂口令策略。
3. ****忘记口令 = 永久解不开****：口令是私钥唯一保护，需企业 KMS/密钥托管方案。
4. ****审计日志权限已限制管理员****（已实现），但尚无角色系统，管理员是扁平列表。

9. 部署指南

9.1 首次部署

```
cd secure-transfer
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt

# 生成生产密钥
openssl rand -hex 32 # → JWT_SECRET

# 生成 TLS 证书（自签，内网）
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
-keyout server.key -out server.crt -subj "/CN=<内网IP或域名>"

# 写 .env
cat > .env <<EOF
JWT_SECRET=<上面生成的hex>
CERT_FILE=./server.crt
KEY_FILE=./server.key
MAX_UPLOAD_MB=2048
ADMIN_USERNAMES=<管理员用户名>
EOF

# 启动
.venv/bin/python run.py
```

9.2 访问

- 本机：`https://127.0.0.1`
- 局域网：`https://<内网IP>`（证书绑定 IP，IP 变化需重签）

9.3 端口说明

- 80 端口：HTTP → HTTPS 自动跳转
- 443 端口：HTTPS 主服务
- 普通用户可绑定 80/443（无需 sudo，但需防火墙放行）

9.4 生产建议

- 用 systemd 或 supervisor 托管进程（而非手动 `python run.py`）
- 定期备份 `data/` 目录（密文 + app.db，丢失则无法解密）
- 替换默认 JWT_SECRET（硬性要求）

- 若有企业 CA，替换自签证书为正式证书

10. 运维指南

10.1 数据位置

数据	位置	说明
-----	-----	-----
SQLite 数据库	<code>data/app.db</code>	用户/文件/密钥/审计
密文文件	<code>data/<uuid>.bin</code>	AES-GCM 密文
WAL 临时文件	<code>data/app.db-wal</code> / <code>app.db-shm</code>	SQLite WAL 模式

10.2 日常操作

- **清空所有数据**：删除 `data/` 目录（密文 + 数据库全清）
- **重启服务**：Ctrl+C 后重新 `python run.py`；改 `.env` 后必须重启
- **健康检查**：`curl -k https://127.0.0.1/healthz` → `{"ok":true}`

10.3 到期清理

后台线程每 60 秒扫描，物理删除过期文件（密文 + files + file_keys）。日志输出 `[purge] 已物理删除 N 个过期文件`。

10.4 审计查询

管理员登录 → 点“审计日志”→ 可搜索 IP/用户名/文件名/动作，支持分页。

11. 性能分析

11.1 实测环境

- 8 核 CPU / 24GB 内存 / 内网部署

11.2 并发下载分析（10 人 × 1GB 同一文件）

维度	结论
-----	-----
内存	安全。FileResponse 流式，不整读内存，每连接仅 ~64KB 缓冲
CPU	几乎空闲。解密在浏览器端，服务端只吐密文
磁盘	page cache 兜底。同文件并发，首个读完后续命中缓存
网络带宽	唯一瓶颈。千兆网 10 人分 ~11MB/s 每人，总时长约 90 秒

11.3 潜在瓶颈与优化方向

1. ****单 worker****：`run.py` 默认 1 进程，建议 10-50 人团队改多 worker（`workers=4`）。
2. ****SQLite 写串行****：审计日志写入在高并发下是小瓶颈，WAL 模式已缓解，几百并发才需担心。
3. ****超大文件****：RSA-OAEP 封装固定 ~256 字节开销，但超大文件建议改「RSA 封装临时对称密钥」分层方案（见 README 设计取舍）。

12. 测试

12.1 端到端集成测试（test_e2e.py）

覆盖 11 项场景，全部通过：

1. 注册（含 RSA 密钥对）
2. 登录
3. AES-GCM 加密上传
4. 文件列表（owner 视角）
5. 未共享用户不可见
6. 共享给他人
7. 接收方下载 + 私钥解封 + 解密，**内容与明文一致**
8. 审计日志记录
9. 撤销访问
10. 过期文件返回 410
11. 删除文件

12.2 文件类型防护测试

- 后端 `<font face="Courier" size="8.5">.exe</font>` 返回 415、`<font face="Courier" size="8.5">.pdf</font>` 返回 200
- 前端 node 模拟 10 场景（evil.pdf/evil.txt/macro.pdf/x.sh.pdf/fake.mp4/fake.zip 拒，真实格式过）
- zip 递归扫描 7 场景（zip 藏 exe/sh/doc 拦、嵌套 zip 藏 exe 拦、合法 zip 过）

12.3 权限与清理测试

- 非管理员访问审计 → 403，管理员 → 200
- 审计搜索/分页
- 过期文件物理删除（含后台线程自动清理）

13. 已知限制与待办

13.1 已知限制

1. ****SQLite 单机****：不适合多服务器分布式部署。
2. ****口令=私钥唯一保护****：忘记口令永久无法解密。
3. ****开放注册****：无邀请制/LDAP/SSO。
4. ****自签证书****：浏览器需手动信任（内网可接受）。

5. **无角色系统**：管理员是扁平用户名列表。

13.2 上线前必做清单

- [] 替换 JWT_SECRET
- [] 启用正式 TLS（或用企业 CA）
- [] 决定用户管理方案（邀请制 / LDAP / SSO）
- [] 设计口令重置与密钥托管（KMS）方案
- [] 提高 Argon2id 迭代次数（t≥10）
- [] 审计日志定期清理/归档策略
- [] systemd/supervisor 托管进程
- [] data/ 目录备份策略

附录 A：动作枚举（audit_log.action）

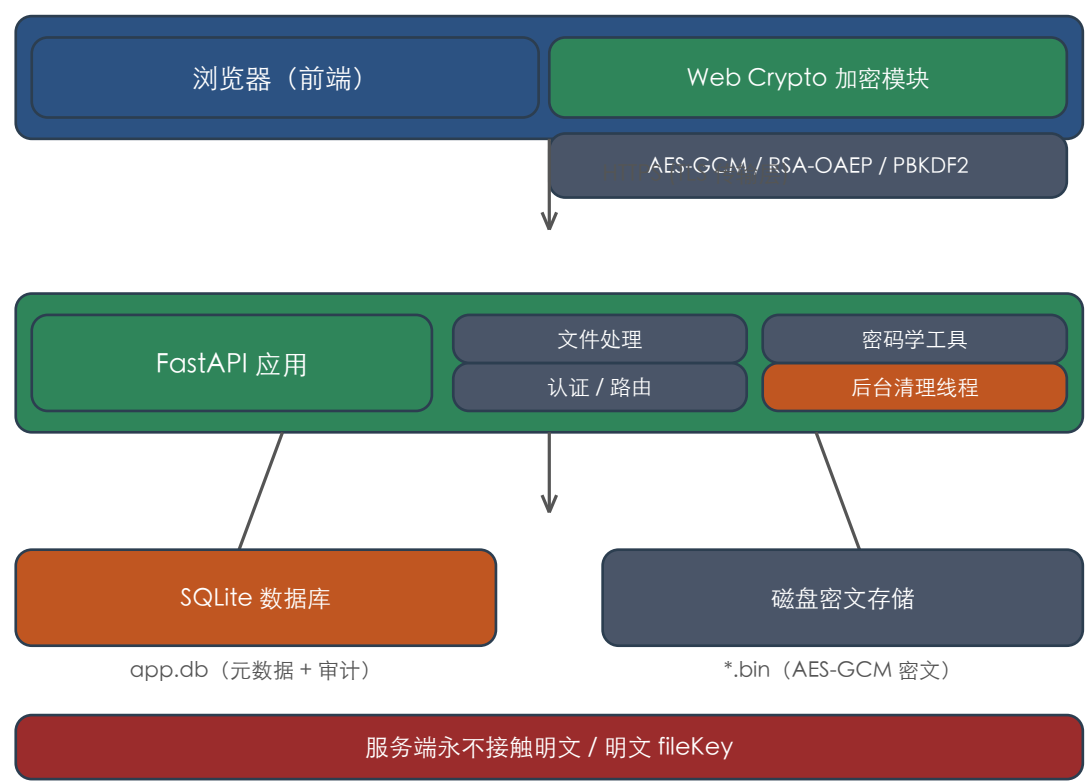
值	含义
----	-----
register	注册
login	登录
upload	上传
download	下载
share	共享
revoke	撤销访问
delete	删除（owner 物理删除）
remove_access	移除访问（非 owner 退出共享）

附录 B：文件类型白名单（18 种）

jpg, jpeg, png, gif, webp, bmp, pdf, docx, xlsx, pptx, txt, csv, md, zip, mp3, wav, mp4, mov

明确排除：`7z`（fflate 不支持解包）、`docm/xlsm/pptm`（宏）、`doc/xls/ppt`（旧二进制）、`svg`（可嵌脚本）、所有脚本/可执行文件。

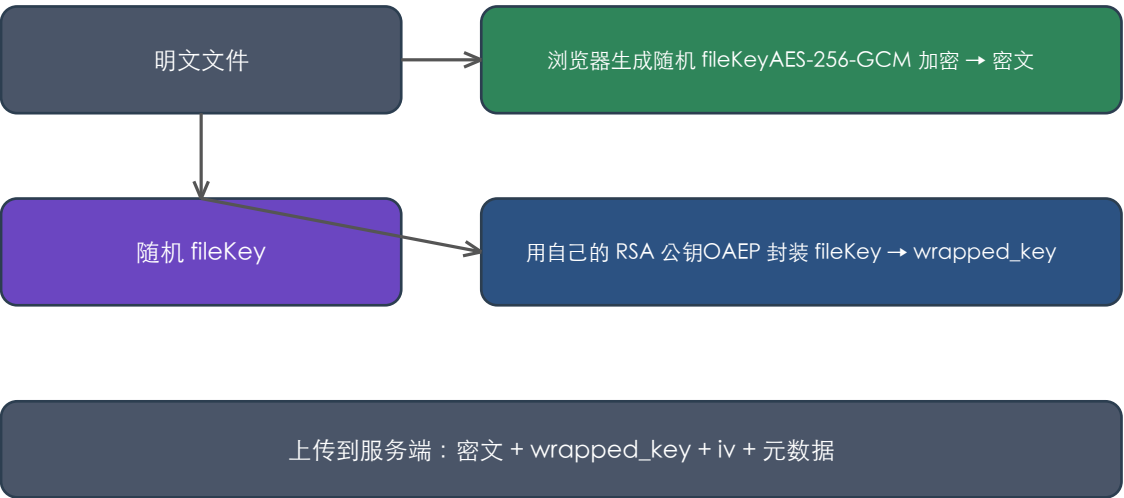
附录 C：系统架构流程图



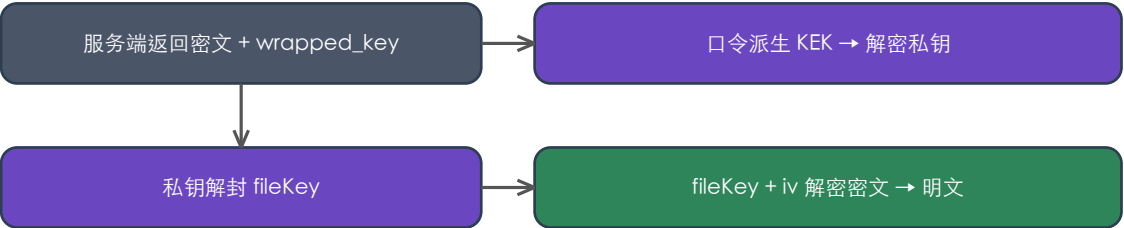
附录 D：E2EE 数据流流程图

端到端加密 (E2EE) 核心数据流

① 上传流程



② 下载流程



③ 多接收方共享

